Krishna Kumar Trivedi Exp-2

➤ Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

➤ Create Dataset

```
data = {
    "Student ID": list(range(1, 31)),   # 30 values

    "Age": [
        18,22,19,21,20,23,24,np.nan,19,20,
        21,22,18,19,20,21,22,23,24,25,
        19,18,20,21,np.nan,22,23,24,19,5
    ],

    "Attendance Percentage": [
        85,90,88,92,75,86,95,89,np.nan,91,
        87,84,98,93,88,85,79,94,96,89,
        91,82,88,98,92,np.nan,85,87,89,30
    ],

    "Study Hours": [
        8,18,12,14,9,11,13,15,7,108,
        12,14,9,8,11,13,15,6,10,12,
        14,9,8,11,13,15,7,10,50,2
    ],

    "Exam Score": [
        78,75,80,85,65,72,88,98,78,np.nan,
        74,76,82,84,79,81,85,60,73,77,
        88,69,71,83,86,88,75,78,95,np.nan
    ]
}

df = pd.DataFrame(data)

df.head()
```

	Student ID	Age	Attendance Percentage	Study Hours	Exam Score
0	1	18.0	85.0	8	78.0
1	2	22.0	90.0	18	75.0
2	3	19.0	88.0	12	80.0
3	4	21.0	92.0	14	85.0
4	5	20.0	75.0	9	65.0

Next steps: Generate code with df New interactive sheet

Check Missing Values

```
# Check number of missing values in each column
df.isnull().sum()
```

	0
Student ID	0
Age	2
Attendance Percentage	2
Study Hours	0
Exam Score	2

dtype: int64

Find Invalid Study Hours (> 20)

```
# Find students having Study Hours greater than 20
df_invalid_study_hours = df[df["Study Hours"] > 20]

df_invalid_study_hours
```

	Student ID	Age	Attendance Percentage	Study Hours	Exam Score
9	10	20.0	91.0	108	NaN
28	29	19.0	89.0	50	95.0

Next steps: Generate code with df_invalid_study_hours New interactive sheet

Fill Missing Values

```
# Fill missing Age with median
df["Age"] = df["Age"].fillna(df["Age"].median())

# Fill missing Attendance Percentage with mean
df["Attendance Percentage"] = df["Attendance Percentage"].fillna(
    df["Attendance Percentage"].mean()
)

# Fill missing Exam Score with mean
df["Exam Score"] = df["Exam Score"].fillna(
    df["Exam Score"].mean()
)

# Check again for null values
df.isnull().sum()
```

	0
Student ID	0
Age	0
Attendance Percentage	0
Study Hours	0
Exam Score	0

dtype: int64

Function to Detect Outliers Using IQR Method

```
def detect_outliers_iqr(series):
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    return lower, upper
```

Detect Outliers for Each Column

```
for col in ["Age", "Attendance Percentage", "Study Hours", "Exam Score"]:
    lower, upper = detect_outliers_iqr(df[col])

    outliers = df[(df[col] < lower) | (df[col] > upper)]

    print("Lower Bound of", col, ":", lower)
    print("Upper Bound of", col, ":", upper)
    print("Outliers:\n", outliers[col])
    print("\n")
```

```
Lower Bound of Age : 14.5
Upper Bound of Age : 26.5
Outliers:
 29    5.0
Name: Age, dtype: float64
```

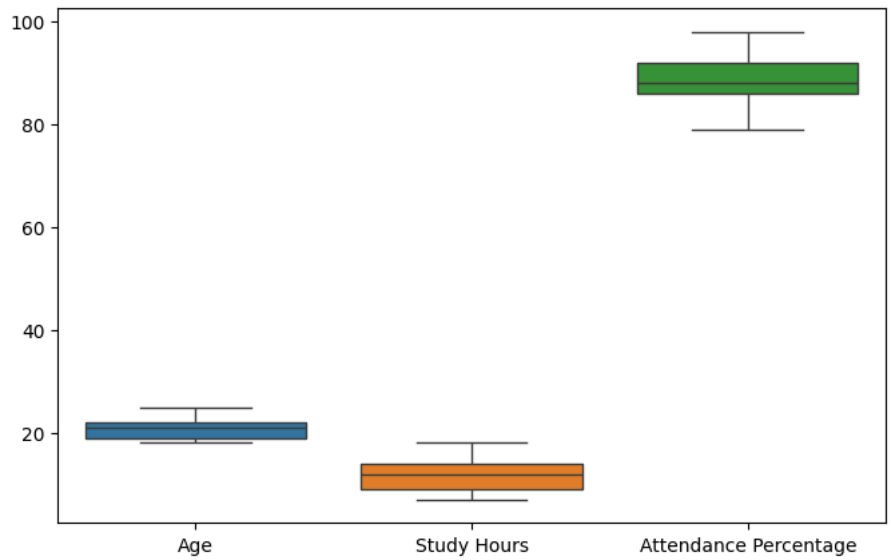
```
Lower Bound of Attendance Percentage : 75.5
Upper Bound of Attendance Percentage : 101.5
Outliers:
 4     75.0
29    30.0
Name: Attendance Percentage, dtype: float64


Lower Bound of Study Hours : 1.5
Upper Bound of Study Hours : 21.5
Outliers:
 9     108
28     50
Name: Study Hours, dtype: int64


Lower Bound of Exam Score : 60.375
Upper Bound of Exam Score : 99.375
Outliers:
 17    60.0
Name: Exam Score, dtype: float64
```

Boxplot Visualization

```
plt.figure(figsize=(8, 5))
sns.boxplot(data=df[["Age", "Study Hours", "Attendance Percentage"]])
plt.show()
```



Remove Outliers (Optional Cleaning Step)

```
for col in ["Age", "Attendance Percentage", "Study Hours", "Exam Score"]:
    lower, upper = detect_outliers_iqr(df[col])
    df = df[(df[col] >= lower) & (df[col] <= upper)]

df.head()
```

	Student ID	Age	Attendance Percentage	Study Hours	Exam Score
0	1	18.0	85.0	8	78.0
1	2	22.0	90.0	18	75.0
2	3	19.0	88.0	12	80.0
3	4	21.0	92.0	14	85.0
5	6	23.0	86.0	11	72.0

Next steps: Generate code with df New interactive sheet

Data Transformation

```
import numpy as np
```

```
def transform_data(df):
    # Log transformation of Study Hours
    df["Log Study Hours"] = np.log(df["Study Hours"])

    # Scale Attendance to 0-1
    df["Attendance Scaled"] = df["Attendance Percentage"] / 100

    return df

df = transform_data(df)

df.head()
```

	Student ID	Age	Attendance Percentage	Study Hours	Exam Score	Log Study Hours	Attendance Scaled
0	1	18.0	85.0	8	78.0	2.079442	0.85
1	2	22.0	90.0	18	75.0	2.890372	0.90
2	3	19.0	88.0	12	80.0	2.484907	0.88
3	4	21.0	92.0	14	85.0	2.639057	0.92
5	6	23.0	86.0	11	72.0	2.397895	0.86

Next steps: Generate code with df New interactive sheet